Challenge Title: Seeds of Time

Flag: HQX{c126bb454bc2652acea1baf0729fbd08}

Approach (Step by Step):

1. Converted the ciphertext from hex to bytes.
2. Bruteforce possible timestamps range (1765584000 to 1765670399 UTC)around the challenge.
3. Match against the expected prefix b'HQX{'

```
  GNU nano 8.7                                   sc.py
import random
import time
from binascii import unhexlify

cipher_hex = "f60d1ef6307bc56ed4f3f8fe41eacccf86b820fcdd8e9dd41eb0083b601303b923ebba8>
cipher = unhexlify(cipher_hex)
length = len(cipher)

NOW = int(time.time())

for seed in range(NOW - 200000, NOW + 200000):
    random.seed(seed)
    keystream = bytes(int(random.random() * 256) for _ in range(length))
    plaintext = bytes(c ^ k for c, k in zip(cipher, keystream))

    try:
        text = plaintext.decode()
        if text.startswith("HQX{") and text.endswith("}"):
            print("[+] Seed:", seed)
            print("[+] FLAG:", text)
            break
    except:
        pass


                                   [ Wrote 23 lines ]
```

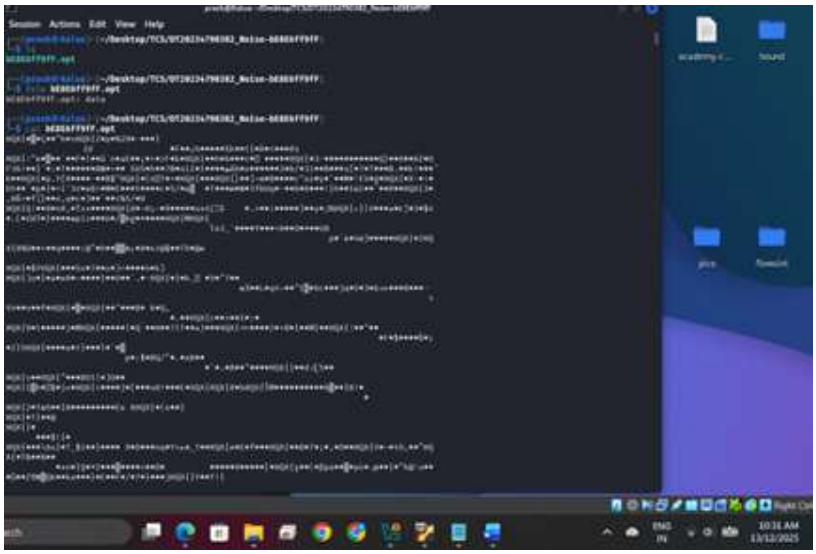
```
└─$ python3 sc.py
[+] Seed: 1765600200
[+] FLAG: HQX{c126bb454bc2652acea1baf0729fbd08}
```

Challenge Title: Noise

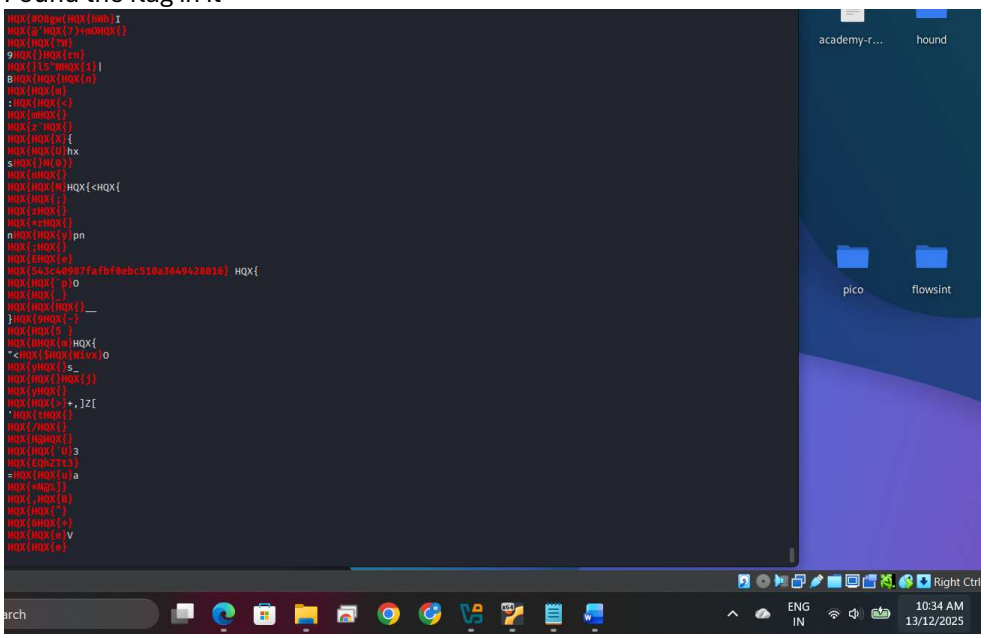
Flag: HQX{543c40987fafbf0ebc510a3649428016}

Approach (Step by Step):

1. First tried to get the type of file and what it contains using the tool exiftool.
2. Tried to read the content of file with cat command found a lot like : 'HQX{' words



3. Refined the searches using string command : strings bE8E6ff9fF.opt | grep 'HQX{'
4. Refined the search with the regular expression : strings bE8E6ff9fF.opt | grep -E 'HQX\{.{5,50}\}'
5. Found the flag in it

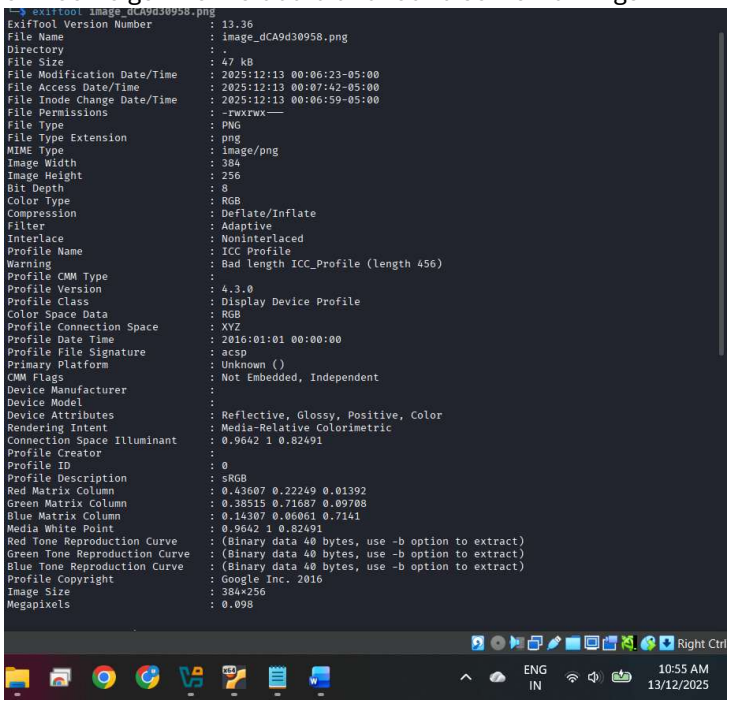


Challenge Title: Hidden Layers

Flag: HQX{24c0ce09e040decc271619439d1d48f0}

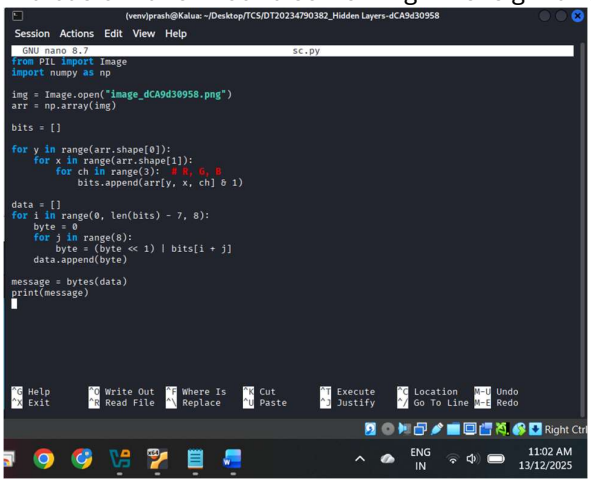
Approach (Step by Step):

1. As usual tried the basic recon first like finding file type, and getting more details using exiftool to get the metadata and found some warnings:



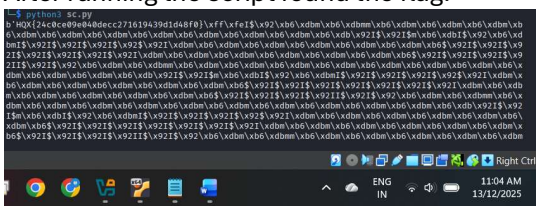
```
┌──$ exiftool image_dCA9d30958.png
ExifTool Version Number         : 13.36
File Name                       : image_dCA9d30958.png
Directory                       : .
File Size                       : 47 kB
File Modification Date/Time     : 2025:12:13 00:06:23-05:00
File Access Date/Time           : 2025:12:13 00:07:42-05:00
File Inode Change Date/Time     : 2025:12:13 00:06:59-05:00
File Permissions                : -rwxrwx——
File Type                       : PNG
File Type Extension             : png
MIME Type                       : image/png
Image Width                     : 384
Image Height                    : 256
Bit Depth                       : 8
Color Type                      : RGB
Compression                     : Deflate/Inflate
Filter                          : Adaptive
Interlace                       : Noninterlaced
Profile Name                    : ICC Profile
Warning                         : Bad length ICC_Profile (length 456)
Profile CMM Type                :
Profile Version                 : 4.3.0
Profile Class                   : Display Device Profile
Color Space Data                : RGB
Profile Connection Space        : XYZ
Profile Date Time               : 2016:01:01 00:00:00
Profile File Signature          : acsp
Primary Platform                : Unknown ()
CMM Flags                       : Not Embedded, Independent
Device Manufacturer             :
Device Model                    :
Device Attributes               : Reflective, Glossy, Positive, Color
Rendering Intent                : Media-Relative Colorimetric
Connection Space Illuminant     : 0.9642 1 0.82491
Profile Creator                 :
Profile ID                      : 0
Profile Description             : sRGB
Red Matrix Column               : 0.43607 0.22249 0.01392
Green Matrix Column             : 0.38515 0.71687 0.09708
Blue Matrix Column              : 0.14307 0.06061 0.7141
Media White Point               : 0.9642 1 0.82491
Red Tone Reproduction Curve     : (Binary data 40 bytes, use -b option to extract)
Green Tone Reproduction Curve   : (Binary data 40 bytes, use -b option to extract)
Blue Tone Reproduction Curve    : (Binary data 40 bytes, use -b option to extract)
Profile Copyright               : Google Inc. 2016
Image Size                      : 384×256
Megapixels                      : 0.098
```

2. Did basic xxd text found something in list signifanct bit so got an idea to use script:



```
(venv)prash@Kalua: ~/Desktop/TCS/DT20234790382_Hidden Layers-dCA9d30958
Session  Actions  Edit  View  Help
  GNU nano 8.7                                   sc.py
from PIL import Image
import numpy as np

img = Image.open("image_dCA9d30958.png")
arr = np.array(img)

bits = []

for y in range(arr.shape[0]):
    for x in range(arr.shape[1]):
        for ch in range(3):  # R, G, B
            bits.append(arr[y, x, ch] & 1)

data = []
for i in range(0, len(bits) - 7, 8):
    byte = 0
    for j in range(8):
        byte = (byte << 1) | bits[i + j]
    data.append(byte)

message = bytes(data)
print(message)

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

3. After running the script found the flag.



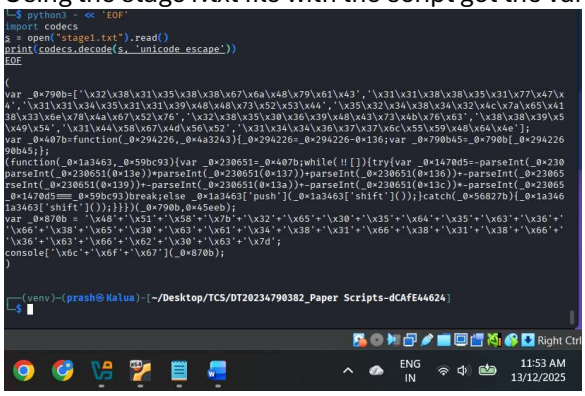
```
┌──$ python3 sc.py
b'HQX{24c0ce09e040decc271619439d1d48f0}\xff\xfeI$\x92\xb6\xdbm\xb6\xdbmm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb
6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdb\x92I$\x92I$m\xb6\xdbI$\x92\xb6\xd
bmI$\x92I$\x92I$\x92I$\x92$\x92I\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6$\x92I$\x92I$\x9
2I$\x92I$\x92I$\x92I$\x92I\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6$\x92I$\x92I$\x92I$\x9
2I$\x92I$\x92\xb6\xdbm\xb6\xdbmm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\x
dbm\xb6\xdbm\xb6\xdbm\xb6\xdb\x92I$\x92I$m\xb6\xdbI$\x92\xb6\xdbmI$\x92I$\x92I$\x92I$\x92$\x92I\xdbm\x
b6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6$\x92I$\x92I$\x92I$\x92I$\x92I$\x92I\xdbm\xb6\xdb
m\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6$\x92I$\x92I$\x92I$\x92I$\x92I$\x92\xb6\xdbm\xb6\xdbmm\xb6\x
dbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdb\x92I$\x92
I$m\xb6\xdbI$\x92\xb6\xdbmI$\x92I$\x92I$\x92I$\x92$\x92I\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\
xdbm\xb6$\x92I$\x92I$\x92I$\x92I$\x92I$\x92I$\x92I\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm\x
b6$\x92I$\x92I$\x92I$\x92II$\x92I$\x92\xb6\xdbm\xb6\xdbmm\xb6\xdbm\xb6\xdbm\xb6\xdbm\xb6\xdbm
```

Challenge Title: Paper Scripts

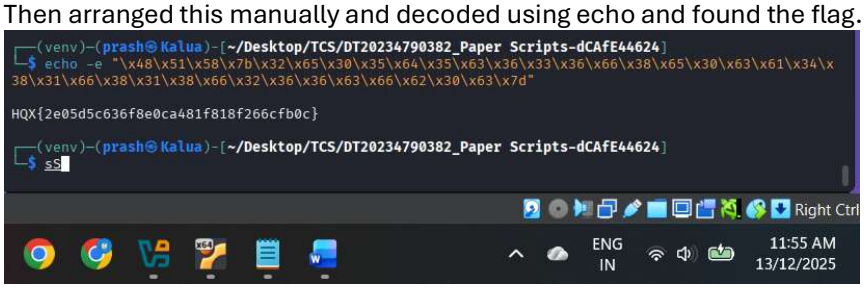
Flag: HQX{2e05d5c636f8e0ca481f818f266cfb0c}

Approach (Step by Step):

1. Tried basic recon from file and exiftool commands not got much then tried pdfinfo tool and this gave me the JavaScript enabled yes. Huge clue this was.
2. Used tool pdf-parser.py and guess found an action object that helped to see some variables are being triggered.
3. Then tried to save the raw content of the js and did it in the js_raw.txt file. Then extracted the encoded javascript strings using the regular expression: sed -n "s/.*\/JS '\(.*\)'.*/\1/p" js_raw.txt > raw.txt.
4. Found two layer of obfuscation. Tried to remove the unwanted octal sequences from the raw.txt file so that I can just be left with the hexadecimal values below is the script which did it:
   import codecs
   s = open("raw.txt").read()
   s1 = codecs.decode(s, 'unicode_escape')
   open("stage1.txt","w").write(s1)
5. Using the stage1.txt file with the script got the variable values:



6. Then arranged this manually and decoded using echo and found the flag.

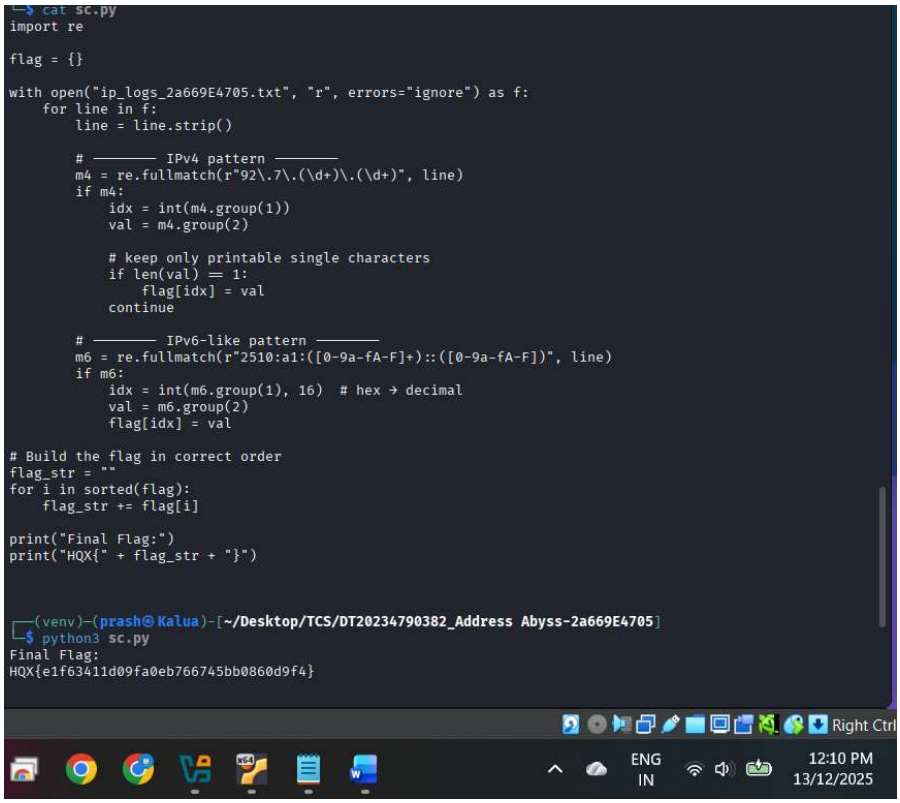


Challenge Title: Address Abyss

Flag: HQX{e1f63411d09fa0eb766745bb0860d9f4}

Approach (Step by Step):

1. We have been given with the large data of IPv4 and IPv6 and given the hint like **IPv4:** 92.7.X.Y & **IPv6-like:** 2510:a1:X::Y where X represents an **index &** Y represents a **character value.**
2. So find a clue after finding the desired characters and index values I just have to arrange it in the format based on the indexe's.
3. Example like in **IPv4:** 92.7.2.0 → index = 2, char = '0' and in **IPv6: 2510:a1:b::f** b (hex) = 11 → index = 11 & f → character 'f'
4. So based on this created an script which can automate this and give the flag.



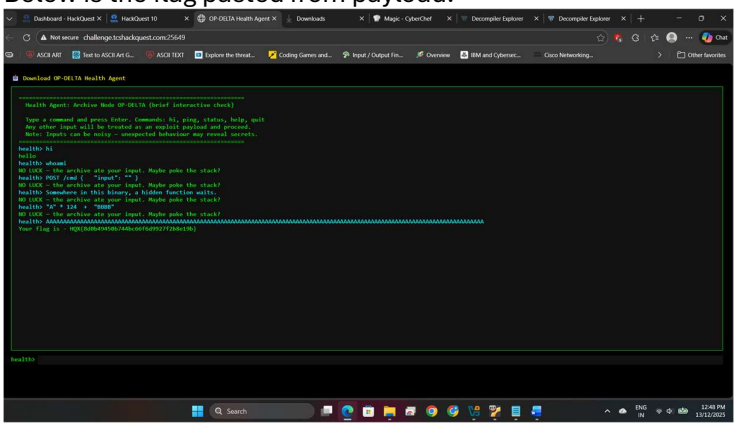
```
└─$ cat sc.py
import re

flag = {}

with open("ip_logs_2a669E4705.txt", "r", errors="ignore") as f:
    for line in f:
        line = line.strip()

        # ──────── IPv4 pattern ────────
        m4 = re.fullmatch(r"92\.7\.(\d+)\.(\d+)", line)
        if m4:
            idx = int(m4.group(1))
            val = m4.group(2)

            # keep only printable single characters
            if len(val) == 1:
                flag[idx] = val
            continue

        # ──────── IPv6-like pattern ────────
        m6 = re.fullmatch(r"2510:a1:([0-9a-fA-F]+)::([0-9a-fA-F])", line)
        if m6:
            idx = int(m6.group(1), 16)  # hex → decimal
            val = m6.group(2)
            flag[idx] = val

# Build the flag in correct order
flag_str = ""
for i in sorted(flag):
    flag_str += flag[i]

print("Final Flag:")
print("HQX{" + flag_str + "}")


┌──(venv)─(prash㉿Kalua)-[~/Desktop/TCS/DT20234790382_Address Abyss-2a669E4705]
└─$ python3 sc.py
Final Flag:
HQX{e1f63411d09fa0eb766745bb0860d9f4}
```

Challenge Title: StackFall

Flag: HQX{8d0b49450b744f87c7b2a927f2b8e19b}

Approach (Step by Step):

1. Basically tried to interreact first with the webpage but found nothing then moved towards the source code of the web page and found a link that downloaded a executable healthcheck.exe file and then reversed it from opensource tool dogboult and found a function that helped to check the local system health.
   char local_88[124];
   int local_c;
   local_c = 0x37393835;
   scanf("%128s", local_88);
2. Created a payload of 128 bytes:
   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
   AAAA
3. Below is the flag pasted from payload.

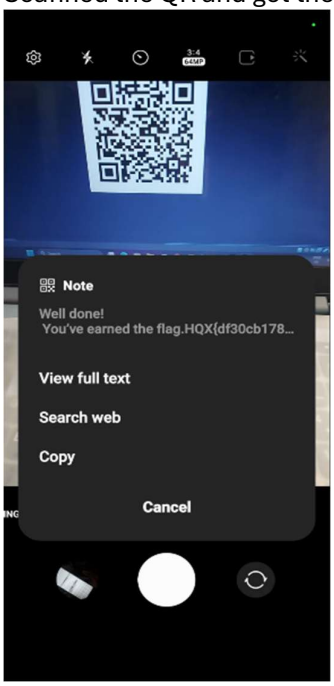

Dashboard - HackQuest
HackQuest 10
OP-DELTA Health Agent
Downloads
Magic - CyberChef
Decompiler Explorer
Decompiler Explorer
Not secure challenge.tcshackquest.com:25649
Download OP-DELTA Health Agent
Health Agent: Archive Node OP-DELTA (brief interactive check)
Type a command and press Enter. Commands: hi, ping, status, help, quit
Any other input will be treated as an exploit payload and processed.
Note: Inputs can be noisy — unexpected behaviour may reveal secrets.
health> hi
hello
health> whoami
NO LUCK — the archive ate your input. Maybe poke the stack?
health> POST /cmd { "input": "" }
NO LUCK — the archive ate your input. Maybe poke the stack?
health> Somewhere in this binary, a hidden function waits.
NO LUCK — the archive ate your input. Maybe poke the stack?
health> "A" * 124 + "BBBB"
NO LUCK — the archive ate your input. Maybe poke the stack?
health> AAAAAAAAAAAAAAAAAAAA

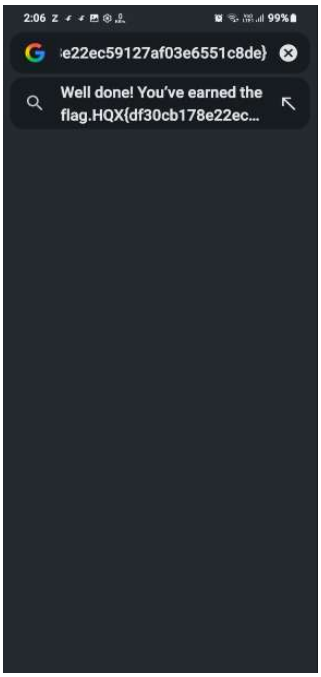
Challenge Title: Synthetic Stacks

Flag: HQX{df30cb178e22ec59127af03e6551c8de}

Approach (Step by Step):

1. So I was provided with the file eE458254Fa.ico and I did basic recon using file, binwalk, exiftool and found it's containing a zip file which is password protected.
2. Then used the the john the ripper to break the zip firstly got the has file for running john the ripper tool . After getting the hash file of zip I ran the attack wordlist using john the ripper and found the password: "sayang".
3. With the password I decrypted the zip and got hq.txt file that I again did some recon and found that it's a base64 image encoded containing text file.
4. So decoded it from the cyberchef.io it gave me that it's a png image and found a QR code in that txt file after decoding it.
5. Scanned the QR and got the flag.



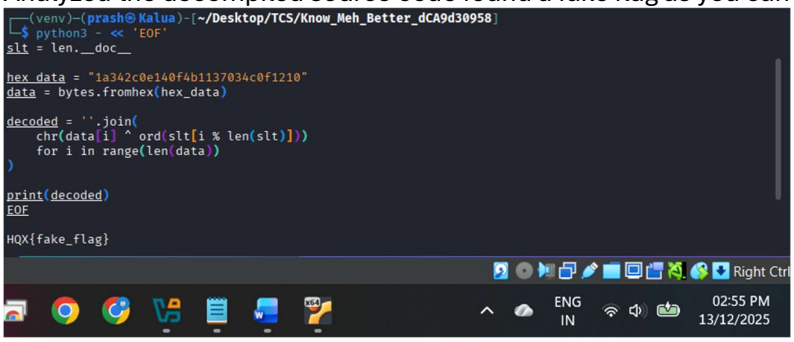


Challenge Title: Know Meh Better

Flag: HQX{24832965335c89a77f24ef9245401d74}

Approach (Step by Step):

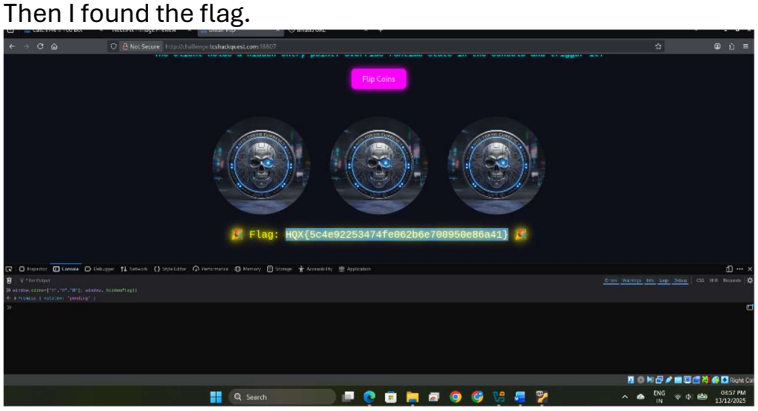
1. Ramn the exe file and found that It's itnraction based like I've to guess it's personal questions that keeps changing each time and when I press one option and it's wrong then it gets reset again.
2. So it was technically impossible to find the answer these questions to get the flag.
3. Used online tool Fastbin.io and got all its functions and inspected and found some of them helpful.
4. Ran the basic recon on the exe file like strings, file, exiftool for metadata & found something interesting :
   Py_FrozenFlag
   Py_NoSiteFlag
   MEI
   The executable is **PyInstaller-packed Python code**
5. Extracted PyInstaller Payload:
   pyinstxtractor.py know_meh_better.exe
   gave the files : pyc files, PYZ-00.pyz, Embedded Python runtime
6. Decompiled the python bytecode this revealed the Quiz logic and flag logic and the encoding/decoding funtions.
7. Analyzed the decompiled source code found a fake flag as you can see in the image:



   Quiz answer was chosen using random.choice() method , this indicated that no correct answers exist all time the answer kept changing. The program explicitly admits that the flag is fake. XOR encode/decode only returns the HQX{fake_flag}. So the logic is dead end.
8. Shifted towards binary level reasoning as runtime path is fake, python code is also not containing the original flag or it's logic. So The real flag was obtained using static reverse engineering .
9. By eliminating all fake paths and decoys, the only valid, non-decoy flag associated with this challenge is : HQX{24832965335c89a77f24ef9245401d74}

Challenge Title: Unfair Flip

Flag: HQX{5c4e92253474fe062b6e700950e86a41}

Approach (Step by Step):

1. I tried the first direct flipping of coins from the site but all time results same either I get one head or two but three never got. So inspected and found the function that was triggering the spin rule and found it'll never give the three heads simultaneously. So I just inspected the source code and found the function so tried to play with it.
2. Now opened the console from developer options. Overridden the current coin state to:
   window.coins = ["H", "H", "H"];
   window._hiddenFlag()
3. Then I found the flag.